Combinatorics

Combinatorics is the study of counting.

But not counting by listing.

Instead:

- Counting using logic
- Counting when numbers are too large to enumerate

Why combinatorics is needed in Competitive Programming

In CP, you will often see:

- n up to 10^5, 10^6, or even 10^9
- Listing all possibilities is impossible

Example

- How many binary strings of length 100000 exist?

You cannot generate them.
But you can count them.

Answer:

2^100000

This is combinatorics.

What kind of CP problems use combinatorics?

- Counting paths
- Counting subsets
- Probability
- String counting
- DP with counting states
- Graph problems
- Bitmask problems

The Foundation: Counting Principles

Everything in combinatorics rests on two rules.

Principle of Product (Multiplication Rule)

If a task has:

- A ways to do part 1
- B ways to do part 2
  and both parts are independent

Then total ways = A × B

Why it works (intuition)

For each way of doing part 1,
you can do part 2 in B ways.

So:

A blocks × B inside each block

Problem

How many strings of length 3 can be formed using digits 0–9?

Thinking

Position 1 → 10 choices

Position 2 → 10 choices

Position 3 → 10 choices

Apply product rule

10 × 10 × 10 = 10^3

Problem

How many binary strings of length n?

Each position:

0 or 1

So:
2 × 2 × 2 × ⋯ × 2 = 2^n

Whenever you see:

- "Choose independently"
- "For each position"
- "For each step"

→ Multiply

Principle of Sum (Addition Rule)

If a task can be done in:

- A ways using method 1
- B ways using method 2
  and the methods are mutually exclusive

Total ways = A + B

Why it works

You either do method 1
or method 2 — never both.

Problem

How many ways to choose:

- an even number from {2,4,6} OR
- an odd number from {1,3}?

Even → 3 ways
Odd → 2 ways

Total:

3 + 2 = 5

Permutations (Order Matters)

A permutation is an arrangement where order matters

Example

Arrangements of {A, B, C}:

ABC
ACB
BAC
BCA
CAB
CBA

Total = 6

Why does order matter?

Because:

ABC ≠ BAC

If swapping positions changes the result → permutation

Deriving n!

How many ways to arrange n distinct elements?

Think step by step

- Position 1 → n choices
- Position 2 → n−1
- Position 3 → n−2

...

Last → 1

Multiply (product rule)

n × (n−1) × (n−2) × ⋯ × 1 = n!

Permutations of r elements from n (nPr)

Choose r elements and arrange them

Step-by-step

- First → n
- Second → n−1
- ...
- r-th → n-r+1

P(n,r) = n × (n − 1) × ⋯ × (n − r + 1)

In factorial form

$$P(n,r) = \frac{n!}{(n-r)!}$$

Problem

How many ways to assign gold, silver, bronze medals among 10 players?

Order matters.

P(10,3) = 10 × 9 × 8

Use permutation when:

- Positions are different
- Rankings
- Arrangements
- Order changes meaning

Combinations (Order Does NOT Matter)

A combination is a selection where order does not matter

Example

Choosing 2 fruits from {A, B, C}:

{A,B} = {B,A} → same

Deriving nCr

Step 1: Count permutations

P(n,r)

Step 2: Fix overcounting

Each combination counted r! times

Divide:

$$C(n,r) = \frac{P(n,r)}{r!} = \frac{n!}{r!(n-r)!}$$

Binomial Coefficient

A binomial coefficient is written as:

$$\binom{n}{r}$$

and it means:

Number of ways to choose r elements from n elements, where order does NOT matter

This is exactly the same as combinations.

So:

$$\binom{n}{r} = C(n,r)$$

Why the name "binomial"?

Because these coefficients appear in the expansion of a binomial:

$$(a+b)^n$$

Example:

$$(a+b)^3 = a^3 + 3a^2b + 3ab^2 + b^3$$

The numbers:

1 3 3 1

are binomial coefficients.

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

This formula is conceptual, not always usable directly in code (factorials get huge).

Key Properties of Binomial Coefficients

1. Symmetry Property

$$\binom{n}{r} = \binom{n}{n-r}$$

Why is this true?

Choosing r elements to include = choosing n - r elements to exclude

CP Usage

- Always compute the smaller of r and n-r
- Reduces time and overflow

2. Boundary Values

$$\binom{n}{0} = 1, \binom{n}{n} = 1$$

Meaning

- Choose nothing → exactly 1 way
- Choose everything → exactly 1 way

3. Pascal's Identity

$$\binom{n}{r} = \binom{n-1}{r} + \binom{n-1}{r-1}$$

Proof

Pick a particular element X.

To choose r elements from n:

Case 1: X is NOT chosen

→ choose r from remaining n - 1

$$\binom{n-1}{r}$$

Case 2: X IS chosen

→ choose remaining r - 1 from n - 1

$$\binom{n-1}{r-1}$$

Add both cases:

$$\binom{n}{r} = \binom{n-1}{r} + \binom{n-1}{r-1}$$

Pascal's Triangle

Structure

Each value is the sum of two values above it.

Row 0: 1
Row 1: 1 1
Row 2: 1 2 1
Row 3: 1 3 3 1
Row 4: 1 4 6 4 1
Row 5: 1 5 10 10 5 1

Why Pascal's Triangle matters in CP

It gives:

- A DP definition of nCr
- A way to compute binomial coefficients without factorials
- A foundation for many counting DP problems

Code: Building Pascal's Triangle (Basic DP)

This is useful when:

- n ≤ 2000 (small constraints)
- No modulo inverses needed

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N = 10;
    vector<vector<long long>> C(N+1, vector<long long>(N+1, 0));

    for (int n = 0; n <= N; n++) {
        C[n][0] = C[n][n] = 1; // boundary cases
        for (int r = 1; r < n; r++) {
            C[n][r] = C[n-1][r] + C[n-1][r-1];
        }
    }

    // Example: print C(5,2)
    cout << C[5][2] << endl; // 10
}
```

This is dynamic programming:

- State: C[n][r]
- Transition: Pascal's identity

Binomial Theorem

$$(a+b)^n = \sum_{r=0}^{n} \binom{n}{r} a^{n-r} b^r$$

Interpretation

- Choose r positions for b
- Remaining n-r positions are a
- Number of such choices = (n r)

Why do we need "nCr modulo prime"?

The problem with the formula

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

In CP:

- n can be up to 10^6, 10^7, sometimes 10^5
- n! grows extremely fast
- Numbers overflow even long long

So we compute:

$$\binom{n}{r} mod 10^9 + 7$$

$$\binom{n}{r} = \text{fact}[n] \times \text{invfact}[r] \times \text{invfact}[n-r] \pmod{p}$$

Full Code

```
#include <bits/stdc++.h>
using namespace std;

static const long long MOD = 1000000007;
static const int MAXN = 1000000;

long long fact[MAXN + 1];
long long invfact[MAXN + 1];

long long modpow(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1) res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

void precompute() {
    fact[0] = 1;
    for (int i = 1; i <= MAXN; i++) {
        fact[i] = fact[i-1] * i % MOD;
    }

    // Fermat inverse of fact[MAXN]
    invfact[MAXN] = modpow(fact[MAXN], MOD - 2);

    for (int i = MAXN; i > 0; i--) {
        invfact[i-1] = invfact[i] * i % MOD;
    }
}

long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invfact[r] % MOD * invfact[n-r] % MOD;
}

int main() {
    precompute();

    cout << nCr(5, 2) << endl;  // 10
    cout << nCr(1000000, 500000) << endl;
}
```